

NITE.

Product Requirements Document

v1.0 · Initial Release

Status	Draft
Version	1.0
Author	Founder
Last updated	May 29, 2026
Target launch	4 weeks from kickoff
Platform	Web app (mobile-first)

One-line pitch

NITE is the Rate My Professors for nightlife — a city-first, community-rated bar and club discovery platform built for the stranger in a new city who has no local friends to ask.

1. Overview

1.1 Problem

When someone arrives in a new city and wants to go out, their options are bad. Yelp is too generic — it treats bars like restaurants and surfaces the same tourist traps. Google Maps has no nightlife-specific context. Reddit takes too long. Asking locals assumes you have locals to ask.

There is no dedicated, community-driven, nightlife-specific platform that answers the three questions a stranger actually needs answered:

- What's the scene like in this city overall?
- Which bar fits my vibe tonight?
- Is it worth going there right now — is it busy, is there a cover, is it worth the Uber?

1.2 Solution

NITE is a mobile-first web app where anyone can search a city, browse bars and clubs by neighborhood and music vibe, see live check-in counts, and read community ratings from people who have actually been there. Users can rate bars they've visited and check in when they're there.

1.3 Target user

Primary

The stranger in a new city — a traveler, someone who just moved, or anyone outside their usual neighborhood — who wants to go out but has no local knowledge and no one to ask. They are searching on their phone, probably at night, and need a fast, trusted answer.

Secondary

Locals who want to discover something new in their own city, or who want to contribute ratings for places they know.

1.4 Non-goals for v1

- No social features — no friend following, activity feed, or seeing who else is checked in
- No venue or promoter tools — no business dashboard, claiming, or paid listings
- No events or ticketing
- No native mobile app — web app only
- No recommendations algorithm — sort by rating and check-in count only
- No moderation tools beyond basic spam prevention

2. Features

2.1 Feature inventory

Feature	Priority	Notes
City landing page	P0	Entry point for all users. Search or auto-detect city.
Neighborhood browse	P0	Breakdown of venues by neighborhood within a city.
Bar profile page	P0	Full venue detail — ratings, tags, stats, reviews.
Rating form	P0	Core data generation. 6 fields, under 60 seconds to complete.
Check-in system	P0	Live crowd signal. Requires Google login. Expires after 4 hours.
Google auth	P0	One-tap login. Required to rate or check in.
Venue search	P0	Search bars by name within a city.
Music / vibe filter	P1	Filter city or neighborhood by genre tags.
Add a venue	P1	User-submitted venues via Google Places autocomplete.
SEO metadata	P0	Server-rendered city and bar pages for Google indexing.
Google Places seeding	P0	Pull venue data programmatically to populate the database.
Verified check-in badge	P1	Reviews from users who checked in get a trust badge.
Live check-in count	P1	Real-time subscriber count via Supabase real-time. Resets after 6am
User profile	P1	Basic profile showing rating and check-in history.
Venue claiming	P2	Owners can claim and manage their venue page.
Promoter tools	P2	Paid promotional features for venues.

2.2 Feature detail

City landing page

The front door of the app. A user arrives either via direct URL ([nite.app/chicago](#)), organic search ("best techno bars Chicago"), or the homepage search bar. The page shows:

- City name, total venue count, total ratings
- Neighborhood grid with venue counts and activity indicators
- Top rated venues list, sorted by overall rating
- Live check-in counts surfaced prominently during evening hours (6pm–4am)
- Music / vibe filter strip (All, EDM/Techno, Hip hop, Live music, Dive bars, Rooftop, Sports)
- Search bar to find a specific venue by name

Bar profile page

The core information surface. Accessible at [nite.app/chicago/fabric-chicago](#). Contains:

- Venue name, address, neighborhood, open/closed status with hours
- Overall star rating (1–5) and total rating count
- Tags: music genre, crowd type, price level, cover charge
- Three stat cards: Price, Would go back %, Cover charge amount
- Live crowd bar — visual indicator of current busyness based on active check-ins
- Check-in button — one tap, requires Google login, increments live count
- Reviews list — sorted by recency, with verified check-in badge where applicable
- Rate this venue CTA

Rating form

Triggered from bar profile page. Six fields designed to be completed in under 60 seconds:

- Overall rating — 1 to 5 stars (required)
- Price — \$ / \$\$ / \$\$\$ selector (required)
- Music — multi-select tags: Techno, Hip hop, House, R&B, Top 40, Live band, Country, Other (required, pick 1+)
- Crowd — multi-select tags: College crowd, Young professionals, Locals, Tourist-heavy, LGBTQ+ friendly, Sports crowd, Late night (required, pick 1+)
- Cover — Yes / No toggle. If yes, amount field appears (required)
- Would you go back? — Yes / No (required)

- Comments — free text, max 280 characters (optional)

Submitted ratings are immediately live. No moderation queue for v1. If a user is checked in at the venue, the review receives a “verified check-in” badge automatically.

Check-in system

Users tap “I’m here right now” on a bar profile page. Requires Google login. On check-in:

- A check-in record is written to the database with a 4-hour expiry timestamp (unsure about this, if resets after 4 hours or at 6am the next day.)
 - If a user checks in at another venue they ‘check-out’ of the bar that they were last at.
- The live check-in count for that venue increments in real time for all viewers
- If the user submits a rating within the same session, the review receives a verified badge
- After 4 hours, the check-in expires and the count decrements

After this, they checkin for the day – they will be allowed to enter a “global” chat with everyone at the bar. This chat will be fresh after each day.

No GPS verification in v1. Trust the user. Fake check-in abuse is a v2 problem.

Venue seeding via Google Places API

Before launching in any city, run a seeding script that calls the Google Places API with queries like "bars in Chicago" and "nightclubs in Chicago" across each neighborhood. For each result, store:

- Name, address, coordinates, place_id, opening hours, phone, website
- Google’s price level (mapped to \$/\$\$/\$\$\$)
- Venue type classification

This gives a fully populated database before the first user arrives. The cold start problem is solved programmatically, not socially.

3. Data model

Five tables in Supabase (Postgres). All IDs are UUIDs.

cities

Field	Type	Description
id	uuid	Primary key
name	text	Display name, e.g. Chicago
slug	text	URL slug, e.g. chicago
lat / lng	float	City center coordinates
venue_count	int	Denormalized count, updated on venue insert
created_at	timestamp	Row creation time

venues

Field	Type	Description
id	uuid	Primary key
city_id	uuid	Foreign key → cities
name	text	Venue display name
slug	text	URL slug within city
address	text	Full street address
neighborhood	text	Neighborhood name
lat / lng	float	Venue coordinates
google_place_id	text	Google Places ID for deduplication
overall_rating	float	Computed average, updated on new rating
total_ratings	int	Denormalized count
price_level	int	1–3 (→ \$ / \$\$ / \$\$\$)
music_tags	text[]	Array of genre tags
crowd_tags	text[]	Array of crowd tags
typical_cover	int	Most commonly reported cover amount in cents
added_by	uuid	User ID if user-submitted, null if seeded
created_at	timestamp	Row creation time

ratings

Field	Type	Description
id	uuid	Primary key
venue_id	uuid	Foreign key → venues
user_id	uuid	Foreign key → users
overall	int	1–5 star rating
price_level	int	1–3
music_tags	text[]	Selected genre tags
crowd_tags	text[]	Selected crowd tags
has_cover	boolean	Cover charge present
cover_amount	int	Cover in cents, null if no cover
would_go_back	boolean	Yes / No
comment	text	Optional free text, max 280 chars
checkin_verified	boolean	True if user was checked in at time of rating
created_at	timestamp	Row creation time

checkins

Field	Type	Description
id	uuid	Primary key
venue_id	uuid	Foreign key → venues
user_id	uuid	Foreign key → users
created_at	timestamp	Check-in time
expires_at	timestamp	created_at + 4 hours
expired	boolean	Set by edge function on expiry

users

Field	Type	Description
id	uuid	Primary key (matches Supabase auth UID)
email	text	From Google OAuth
google_id	text	Google subject ID
display_name	text	From Google profile

avatar_url	text	From Google profile
home_city	text	Optional, user-set
rating_count	int	Denormalized
checkin_count	int	Denormalized
created_at	timestamp	Account creation time

4. Tech stack

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router)	SSR for SEO, API routes, React for UI
Database	Supabase (Postgres)	Real-time subscriptions, built-in auth, generous free tier
Auth	Supabase Auth + Google OAuth	One-tap login, no password forms
Venue data	Google Places API	Programmatic city seeding, autocomplete for add-venue
Real-time	Supabase Realtime	Live check-in counts without polling
Hosting	Vercel	Zero-config Next.js deploy, edge network, free tier
Analytics	Posthog	User flow tracking, funnel analysis, free up to 1M events/mo
Serverless	Supabase Edge Functions	Check-in expiry, rating aggregation triggers

4.1 Key architectural decisions

Server-side rendering (SSR) on city and bar pages

City pages (`nite.app/chicago`) and bar profile pages (`nite.app/chicago/fabric-chicago`) are rendered server-side via Next.js. This means Google can crawl and index them immediately. Organic search is the primary acquisition channel — someone searching "best techno bars Chicago" at 10pm is the exact user this product is built for. SSR is non-negotiable.

Real-time check-in counts via Supabase Realtime

When a user checks in, the `checkins` table is updated. All other users viewing that bar's profile page are subscribed to a Supabase channel for that `venue_id`. The count updates in their browser instantly without a page refresh. This creates the "live" feel that differentiates NITE from a static directory.

Denormalized rating averages

Rather than computing `AVG(overall)` on every page load, `overall_rating` and `total_ratings` are denormalized columns on the `venues` table, updated by a Supabase database trigger whenever

a new rating is inserted. This makes the city landing page fast — no joins required to sort venues by rating.

No GPS verification on check-ins

Requiring GPS verification adds meaningful friction, fails on poor connections, and introduces permissions complexity. For v1, trust the user. If check-in gaming becomes a problem at scale, geofencing can be added as a query parameter confirmation step in v2.

5. Build plan

Four weeks from kickoff to first live city. One developer. No external dependencies other than APIs listed in the tech stack.

Week	Milestone	Deliverables
Week 1	Foundation	Supabase schema created. Google Places seeding script written and run for launch city. City page and bar profile page rendering real data from database. No auth, no interactivity — read-only.
Week 2	Core interaction	Google OAuth login via Supabase Auth. Rating form built and wired to database. Ratings display on bar profile. <code>overall_rating</code> computed via trigger.
Week 3	Check-in + live	Check-in system built. Supabase Realtime subscription on bar profile page. Live count updates in real time. Verified check-in badge on eligible reviews. Neighborhood filter and music/vibe filter on city page.
Week 4	Polish + ship	SEO metadata and Open Graph tags on all pages. Add a venue flow with Google Places autocomplete. Mobile polish pass. Posthog analytics installed. Deploy to Vercel. Launch.

5.1 Launch strategy

NITE launches in a single city. The launch city should be one the founder knows well — ideally their home city or a city they go out in regularly — so they can validate that the venue database is accurate and the ratings feel real.

The database is pre-populated via Google Places API before launch. The goal is for the first person who opens NITE to find at least 50–100 venues in their city already listed, with neighborhoods populated. The app should never feel empty on day one.

Growth from there is SEO-driven. Every city page and bar profile page is a crawlable, indexable URL. As the venue database grows and ratings accumulate, Google surfaces more pages. A user searching "best dive bars Austin" on a Friday night is the acquisition channel. Paid acquisition is a v2 consideration.

6. Success metrics

For a side project in its first month, success is not measured in revenue or viral growth. It's measured in signal — evidence that the core loop works and people find it useful.

Metric	Target (month 1)	Why it matters
Venues in launch city	100+	Database can't feel thin
Total ratings submitted	50+	Proves the rating loop works
Unique users	200+	Proves discoverability
Check-ins	25+	Proves the live feature has value
Return visits (same user, 2+ sessions)	20%+	Proves habit formation
Avg session duration	2+ minutes	Proves content is engaging
Google-indexed pages	50+	SEO working

6.1 What kills this product

There are two failure modes worth naming explicitly so they can be watched for early:

- Empty cities. If someone opens NITE in a city and finds fewer than 20–30 venues, they leave and never come back. The Google Places seeding must be run before any city is made discoverable.
- Thin reviews. A bar profile with 0–1 reviews is not useful. The rating form must be easy enough that early users actually fill it out. If the first 20 users each rate 2–3 bars, the core pages become credible quickly.

6.2 Open questions

- How is the first city chosen? Recommend the founder's home city for accuracy validation.
- How are neighborhoods defined? Use Google Places' neighborhood data as a starting point, manually corrected per city.
- What is the check-in expiry? 4 hours is the current assumption. Validate against typical night-out duration.
- How are spam ratings handled in v1? Flag-based user reporting is the minimum. Moderation dashboard is v2.

- What happens when a venue closes permanently? Soft-delete with a “Permanently closed” badge, sourced from Google Places status.

NITE. · Product Requirements Document · v1.0 · Confidential